

# Learning to Hash for Efficient Search over Incomplete Knowledge Graphs

Meng Wang<sup>\*†</sup>, Haomin Shen<sup>‡</sup>, Sen Wang<sup>§</sup>, Lina Yao<sup>¶</sup>, Yinlin Jiang<sup>\*</sup>, Guilin Qi<sup>\*†</sup> and Yang Chen<sup>\*</sup>

<sup>\*</sup>School of Computer Science and Engineering, Southeast University, Nanjing, China

<sup>†</sup>MOE Key Laboratory of Computer Network and Information Integration, China

meng.wang@seu.edu.cn, yljia@seu.edu.cn, gqi@seu.edu.cn, chenyang.list@seu.edu.cn

<sup>‡</sup>School of Cyber Science and Engineering, Southeast University, Nanjing, China

haomin\_shen@seu.edu.cn

<sup>§</sup>The University of Queensland, Brisbane, Australia

sen.wang@uq.edu.au

<sup>¶</sup>The University of New South Wales, Sydney, Australia

lina.yao@unsw.edu.au

**Abstract**—Knowledge graph (KG) embedding techniques represent entities and relations as low-dimensional, continuous vectors, and thus enables machine learning models to be easily adapted to KG completion and querying tasks. However, learned dense vectors are inefficient for large-scale similarity computations. Learning-to-hash is to learn compact binary codes from high-dimensional input data and provides a promising way to accelerate efficiency by measuring Hamming distance instead of Euclidean distance or dot-product. Unfortunately, most of learning-to-hash methods cannot be directly applied to KG structure encoding. In this paper, we introduce a novel framework for encoding incomplete KGs and graph queries in Hamming space. To preserve KG structure information from embeddings to hash codes and address the ill-posed gradient issue in optimization, we utilize a continuation method with convergence guarantees to jointly encode queries and KG entities with geometric operations. The hashed embedding of a query can be utilized to discover target answers from incomplete KGs whilst the efficiency has been greatly improved. We compared our model with state-of-the-art methods on real-world KGs. Experimental results show that our framework not only significantly speeds up the searching process, but also provides good results for unanswerable queries caused by incomplete information.

**Keywords**—knowledge graph embedding, learning to hashing, graph query

## I. INTRODUCTION

Large-scale knowledge graphs (KGs) such as Wikidata [1] and Freebase [2] describe knowledge as triple facts in the form of (*head entity*, *relation*, *tail entity*). Although increasing KGs on the web offer a plethora of structured data sources where users discover items of interest by issuing graph queries, the empty-answer problem [3], or the so-called unanswerable query [4], becomes more serious because most of the KGs suffer from incompleteness. Thus, increasing research attention has been paid into KG completion, i.e., automatically inferring missing facts. In recent years, KG embedding [5] has been proposed to encode entities and relations into a low-dimensional, continuous vector space. The learned dense vector representations, a.k.a. embeddings, of entities and relations mathematically support various machine learning models

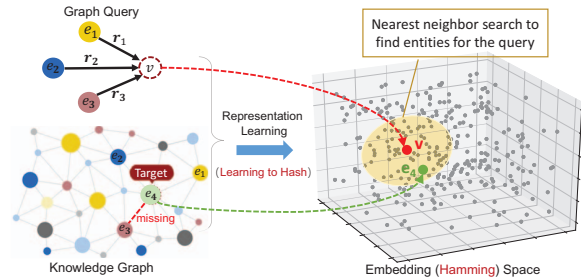


Fig. 1. Searching incomplete KGs in the embedding space.

to perform KG completion, which has demonstrated good performance and robustness, in turn, can be used to improve other tasks, as well as graph querying over incomplete KGs.

Wang et al. [3] and Hamilton et al. [4] have explored different ways to utilize KG embeddings for the query answering, especially for handling the unanswerable queries over incomplete KGs. They have one thing in common: introducing a new search paradigm that answers a graph query by nearest neighbor search in an embedding space. As shown in Figure 1, they predict the embedding of the query target ( $v$ ) by utilizing the embeddings of existing entities ( $e_1, e_2, e_3$ ) and relations ( $r_1, r_2, r_3$ ). Then, the nearest neighbor search for the predicted variable embedding ( $v$ ) can be performed in the embedding space to find the expected answer ( $e_4$ ). Even if missing triples in KGs may make the original query fail in conventional graph matching, the query target still can be predicted in the embedding space.

Leveraging KG embeddings is a promising pathway to answer graph queries over incomplete KGs, but it is also troubling. One of the biggest challenges is that learned dense vectors are inefficient for similarity computations when the dimensionality exceeds 100 approximately [6]. That is, the embeddings are definitely low-dimensional (e.g., 128 or 256) compared the adjacent matrix representation of KG entities, but still barely “high-dimensional” for the nearest neighbor search. This drawback impedes the widely use of KG embedding models for searching real-world KGs.

Hashing is a useful method for fast similarity search with the goal of transforming a data item to a compact binary code [7]. The Euclidean or dot-product similarity calculation in continuous vector space then can be replaced by fast Hamming distance computation. Recently, **learning-to-hash** methods [7], [8] attract enormous research efforts due to their better performance than **data-independent hashing** methods (e.g, locality sensitive hashing [9]). Learning-to-hash is data-dependent and learns hash functions from the input dataset so that the nearest neighbor search result in Hamming space is as close as possible to the search result in the original space. Most of learning-to-hash methods focused on similarity preserving and large scale image search/classification [10], [11], but cannot be simply applied to KGs which have important structure information. Several new models [12]–[14] learn binary codes of entities in homogeneous and heterogeneous networks. However, these models are not applicable to KGs and graph query answering because they only support one or two types of relations encoding and ignore graph queries during hash learning.

Based on the above observations, in this paper, we introduce a novel framework for searching incomplete KGs in Hamming space. We design an end-to-end learning-to-hash model for encoding KG graph queries and entities. To preserve the triple structure information, we define two geometric operators as [3], [4] during binary embeddings learning and query targets prediction. To capture the semantic similarity between entities, we incorporate KG ontology information into the entity embeddings. By jointly encoding queries and entities with geometric operations, the target entity embedding can be approximated with hashed query representation maintaining high-quality searching results and speed-up performance. Moreover, to address the ill-posed gradient issue in the optimization, we utilize a continuation method with convergence guarantees to learn the final compact binary codes.

Our framework makes the following contributions:

- To the best of our knowledge, we are the first to encode incomplete KGs and graph queries in a Hamming space, which speeds up the expected answers discovering for unanswerable KG query problem.
- We propose a novel end-to-end learning-to-hash model which preserves triple structure and ontology information to learn binary embeddings for queries and entities. The model successfully approximates the target entity with query hashing codes.
- To address the ill-posed problem, we propose an efficient algorithm with a continuation method to jointly encode queries and KG entities, and then utilize the learned binary embeddings to find approximate query answers from incomplete KGs.
- We conducted comparative experiments on real-world KGs to evaluate the performance of our framework. Results show that the framework significantly speeds up the searching process and provides good results.

## II. RELATED WORK

This section discusses existing related research in the following aspects: KG embedding techniques, learning-to-hash models and KG approximate search approaches.

To address the incompleteness problem, KG embedding techniques have been proposed to embed both entities and relations into low-dimensional, continuous vector spaces. Among these methods, neural-embedding-based [15] models focus on using neural networks to generate entity latent representations. Tensor-factorization-based [16] and translation-based [17] models are proposed to model relations between entities. The detailed review on KG embedding can be found in [5]. However, dense continuous vectors are inefficient for large-scale similarity computations, and this hinders direct exploitation of existing embedding models on real-world KG query answering.

Learning-to-hash [7] provides a powerful data-dependent method for fast similarity search by learning hash functions from the input dataset. Most of existing learning-to-hash methods focused on pairwise similarity preserving and image research areas [10], [11] and cannot capture KG triple structure information adequately. Recently, some models [12]–[14] learn binary embeddings of entities in homogeneous and heterogeneous networks. However, directly employing these models on KG searching will not obtain good quality. The reason is that graph queries are ignored during the optimization process of embedding learning.

KG approximate search methods have been proposed to solve the empty-answer/unanswerable query problem by fixing the original query into a new relaxed query [18]. However, relaxed queries generated by KG approximate search approaches may be rather different from the original queries because they cannot consider the expected answers which do not occur in search results or missing triples in KGs [3]. Over-relaxed queries will return irrelevant answers which are not useful for meeting the query intention.

New approaches [3], [4] achieve progress by leveraging KG embedding models in the incomplete KG searching task. They aim to predict the expected answers for a given query by nearest neighbor search in embedding space, even if this query cannot be answered directly because of missing triples in KGs. However, the key obstacle of inefficient large-scale similarity calculation between dense vectors impedes the impact of these models on real-world KGs.

## III. PROPOSED METHOD

Before demonstrating the details of our framework, we briefly introduce the notations used in this paper.

**Knowledge Graph.** Let  $\mathcal{E}$  be a set of entities,  $\mathcal{R}$  be a set of relations. A KG  $\mathcal{G} = (\mathcal{E}, \mathcal{R})$  is a finite set of KG triples in the form  $\langle e_h, r, e_t \rangle$ , where  $e_h, e_t \in \mathcal{E}$  and  $r \in \mathcal{R}$ . A KG triple  $\langle e_h, r, e_t \rangle$  indicates a directed relation  $r$  from the head entity  $e_h$  to the tail entity  $e_t$ , e.g.,  $\langle \text{Fight\_Club}, \text{starring}, \text{Brad\_Pitt} \rangle$ .

**Hierarchical KG Ontology.** In addition, the types of entities contain rich information. Given an entity  $e$  in a KG, it may belong to different types, denoted as  $\{c_1, c_2, \dots, c_m\}$ .

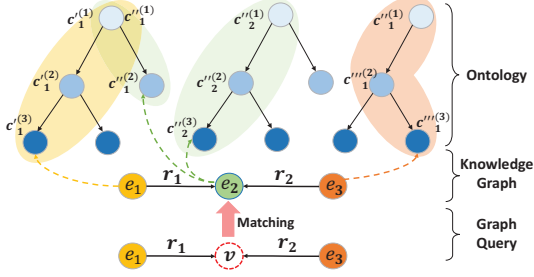


Fig. 2. Illustrations for notations.

Moreover, **each type has only one parent type in the ontology**. For a type  $c_i \in \{c_1, c_2, \dots, c_m\}$ , we further define the hierarchical types set  $C_i$  as  $\{c_i^{(k)}, c_i^{(k-1)}, \dots, c_i^{(1)}\}$ , where  $k$  indicates the layer of  $c_i$  in the hierarchical structure, and each type  $c_i^{(j-1)}$  is the parent of type  $c_i^{(j)}$  ( $1 < j \leq k$ ).

**Graph Query.** Let  $\mathcal{V}$  be a set of entity variables. Each variable  $v \in \mathcal{V}$  is distinguished by a leading question mark symbol, e.g.,  $?film$ . A triple pattern is similar to a KG triple but allows the usage of variables for entities, e.g.,  $\langle ?film, starring, Brad\_Pitt \rangle$ . A graph-structured query  $q = (\mathcal{E}_q \cup \mathcal{V}_q, \mathcal{R}_q)$  is a finite set of triple patterns, where  $\mathcal{E}_q \subseteq \mathcal{E}$ ,  $\mathcal{V}_q \subseteq \mathcal{V}$  and  $\mathcal{R}_q \subseteq \mathcal{R}$ . In the variables set  $\mathcal{V}_q$ , there will be a target variable which indicates the query intention.

Figure. 2 provides a better understand of above notations.

#### A. Problem Statement

In this paper, we define the new search paradigm to evaluate  $q$  in the Hamming space of KG  $\mathcal{G}$  as follows:

Given a KG  $\mathcal{G} = (\mathcal{E}, \mathcal{R})$ , we use bold letter  $\mathbf{e}$  to denote the embedding of  $e \in \mathcal{E}$ . We utilize the information of the triple structure and entity hierarchical types to learn a binarized embedding  $\mathbf{e}$ , i.e.,  $h_e : e \mapsto \{+1, -1\}^d$ , where  $d$  is the dimension of the Hamming space.

For a graph query  $q$ , we aim to learn a hash function  $h_q : q \mapsto \{+1, -1\}^d$  to generate the binary representation of the target variable in  $q$ , and then perform nearest neighbor search (NNS) in the Hamming space to find the entity most  $e$  likely satisfirf this query.

To obtain good search results, the learned binary codes need to satisfy the following two requirements: (1) **low dimensionality**, the dimension of the compact binary code should be much smaller than the dimension of entity adjacent matrix  $|\mathcal{E}|$ ; (2) **high accuracy**, the embedding of target answer should be as closer to the query embedding  $\mathbf{q}$  as possible in the Hamming space.

Given the problems and requirements, our framework includes two phases, as shown in Figure 3: 1) a joint end-to-end learning-to-hash model to encode the KG entities and graph queries into a Hamming space; 2) search incomplete KGs in Hamming space with the predicted query embedding.

#### B. Learning to Hash Model

Given a KG  $\mathcal{G} = (\mathcal{E}, \mathcal{R})$  and a query  $q$ , the embedding  $\mathbf{q}$  should be close to the embedding  $\mathbf{e}$  of an expected query answer  $e \in \mathcal{E}$  in the Hamming space. Because the Hamming distance and dot product can be interchanged by  $\text{dist}_H(\mathbf{q}, \mathbf{e}) =$

$\frac{1}{2}(d - \mathbf{q} \cdot \mathbf{e})$ , we can define the likelihood that  $e$  is one of the query  $q$ 's answers as:

$$\text{score}(\mathbf{q}, \mathbf{e}) = \frac{\mathbf{q} \cdot \mathbf{e}}{\|\mathbf{q}\| \|\mathbf{e}\|}. \quad (1)$$

Thus, our goal is to obtain query embedding  $\mathbf{q}$  so that  $\text{score}(\mathbf{q}, \mathbf{e}) = 1$  for each  $e \in \llbracket q \rrbracket$ , where  $\llbracket q \rrbracket$  represents the set of  $q$ 's search results over  $\mathcal{G}$ . As shown in Figure 3, we learn the binary embeddings by performing the sign function  $\text{sgn}(\cdot)$  as activation function on the  $d$ -dimensional continuous representation as follows:

$$\text{sgn}(x) = \begin{cases} +1, & \text{if } x \geq 0, \\ -1, & \text{otherwise.} \end{cases} \quad (2)$$

Next, we present the details of how to embed the queries and KG entities with triple structure and entity hierarchical types information, as well as the optimization process.

1) **Graph Query Embeddings:** Based on variables shared by triple patterns in a query, we follow the work in [4] and consider 7 types of directed acyclic graph (DAG) queries.

For the projection operator  $\mathcal{P}$ , it outputs the embedding of  $v$  in a triple pattern  $\langle h, r, v \rangle$  based on the given embeddings of  $h$  and  $r$ . This operator can be implemented by any KG embedding-based relation prediction model. For instance, if we use the translation-based KG embedding models, e.g., TransE [17], the projection operator  $\mathcal{P}$  is defined as

$$\mathbf{v} = \mathcal{P}(r, h) = \text{sgn}(\mathbf{h} + \mathbf{r}), \quad (3)$$

where  $\mathbf{r} \in \mathbb{R}^d$  and  $\mathbf{h} \in \{+1, -1\}^d$ . Considering a chained graph query  $q = \{\langle e_1, r_1, v_1 \rangle, \langle v_1, r_2, v_2 \rangle\}$  where  $v_2$  is the target variable, the projection operator  $\mathcal{P}(e, r)$  will outputs a query embedding  $\mathbf{q} = \mathbf{v}_2 = \text{sgn}(\text{sgn}(\mathbf{e}_1 + \mathbf{r}_1) + \mathbf{r}_2)$  which indicates the embedding of the set of entities by evaluating  $q$  over KG, i.e.,  $\llbracket q \rrbracket$ .

The intersection operator  $\mathcal{I}$  is utilized to predict the embedding of a variable in the tail position and shared by multiple triple patterns. Given a query  $q = \{\langle e_1, r_1, v \rangle, \langle e_2, r_2, v \rangle, \dots, \langle e_n, r_n, v \rangle\}$ , we first perform projection operator on each triple pattern and obtain a set of embeddings  $\{\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_n\}$ . Then, the intersection operator  $\mathcal{I}$  takes input as these embeddings and performs set intersection in the Hamming space, i.e., outputs the final embedding  $\mathbf{v}$  which corresponds to  $\llbracket v \rrbracket = \llbracket \langle e_1, r_1, v \rangle \rrbracket \cap \dots \cap \llbracket \langle e_n, r_n, v \rangle \rrbracket$ . Here, we implement  $\mathcal{I}$  as follows:

$$\begin{aligned} \mathbf{v} &= \mathcal{I}(\{\mathbf{v}_1, \dots, \mathbf{v}_n\}) \\ &= \text{sgn}(\mathbf{W}_v \Psi(\{\text{ReLU}(\mathbf{M}_{v_i} \mathbf{v}_i), \forall i = 1, \dots, n\})), \end{aligned} \quad (4)$$

where  $\mathbf{W}_v \in \mathbb{R}^{d \times d}$  is a directly trainable transformation matrix by the neural network for the type of target  $\mathbf{v}$ .  $\mathbf{M}_{v_i} \in \mathbb{R}^{d \times d}$  is the trainable specific matrix for the type of each  $\mathbf{v}_i$ .

For a more complex DAG graph query, we can generate the query embedding  $\mathbf{q}$  by  $\mathcal{P}$  and  $\mathcal{I}$ . Next, we introduce the entity embedding initialization.

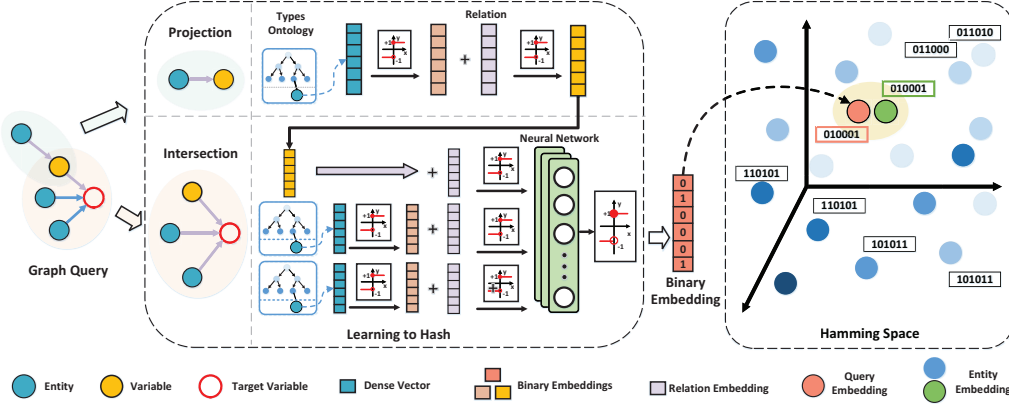


Fig. 3. Overview of our framework.

2) *KG Entity Embedding*: Given an entity  $e$  and its types  $\{c_1, c_2, \dots, c_m\}$ , we follow [19] define a projection matrix  $\mathbf{M}_e$  for entity hierarchical types encoding as:

$$\mathbf{M}_e = \alpha_1 \mathbf{M}_{c_1} + \alpha_2 \mathbf{M}_{c_2} + \dots + \alpha_m \mathbf{M}_{c_m}, \quad \sum_{i=1}^m \alpha_i = 1, \quad (5)$$

where  $m$  is the number of the entity  $e$ 's types,  $c_i \in \{c_1, c_2, \dots, c_m\}$  is the  $i$ -th type,  $\mathbf{M}_{c_i}$  is the projection matrix of  $c_i$ ,  $\alpha_i$  is the weight of  $i$ -th type and can be computed according to the type contributions [19].

For a type  $c_i \in \{c_1, c_2, \dots, c_m\}$  of  $e$ , we can obtain its ancestors set  $C_i = \{c_i^{(k)}, c_i^{(k-1)}, \dots, c_i^{(1)}\}$  by a bottom-up walk. We further adopt weighted hierarchy encoder (WHE) proposed in [19] to consider different weights of types in  $C_i$ . Specifically, we represent each type  $c_i^{(j)} \in C_i$  as a projection matrix and sum up all projection matrices with different weights for encoding the hierarchical  $C_i$  as follows:

$$\mathbf{M}_{c_i} = \sum_{t=1}^k \beta_t \mathbf{M}_{c_i}^{(t)} = \beta_1 \mathbf{M}_{c_i}^{(1)} + \dots + \beta_k \mathbf{M}_{c_i}^{(k)}, \quad (6)$$

where  $\mathbf{M}_{c_i}^{(t)}$  is the projection matrix of  $c_i^{(t)}$ ,  $\beta_t$  is the corresponding weight of  $c_i^{(t)}$  and can be computed by the proportional-declined weighting strategy. Finally, we define the entity embedding of  $e$  as follows:

$$\mathbf{e} = \text{sgn}\left(\frac{\mathbf{M}_e \cdot \mathbf{x}_e}{|\mathbf{x}_e|}\right), \quad (7)$$

where  $\mathbf{M}_e$  is the trainable embedding matrix corresponding to Equation (5),  $\mathbf{x}_e \in \mathbb{R}^d$  is a  $d$ -dimensional continuous representation for  $e$  and is trainable. After applying the sign function, we will initialize the binary embedding  $\mathbf{e} \in \{+1, -1\}^d$ .

3) *Learning and Optimization*: Given a graph query  $q$ , we utilize stochastic gradient descent (SGD) to learn the binary embedding  $\mathbf{q}$  by a DAG graph traversal and topological sort from the given entity embeddings to the query target variable with two geometric operators  $\mathcal{P}$  and  $\mathcal{I}$ . The max-margin loss function is defined as:

$$\mathcal{L}(q) = \max(0, 1 - \text{score}(\mathbf{q}, \mathbf{e}^+) + \text{score}(\mathbf{q}, \mathbf{e}^-)), \quad (8)$$

in which  $\mathbf{e}^+ \in \llbracket q \rrbracket$  is the positive instance entity,  $\mathbf{e}^- \notin \llbracket q \rrbracket$  is the negative instance in the training set. We uniformly sample positive and negative instances to train embedding  $\mathbf{q}$ .

To learn the binary embeddings, we employ sign function as activations. For the intersection operator in Equation (4), a deep  $k$ -layer neural network will be constructed for an expressive learning. Therefore, the ill-posed gradient difficulty with sign activations will be a problem in the optimization because the derivative of sign function is zero almost everywhere. Inspired by the continuation method [8], we approximate the non-smooth sign function  $\text{sgn}(x)$  with the scaled tanh function  $\tanh(x)$  in terms of the following property:

$$\lim_{\delta \rightarrow \infty} \tanh(\delta x) = \text{sgn}(x), \quad (9)$$

in which  $\delta > 0$  is a scaling parameter. The tanh function  $\tanh(x)$  will converge to the binary embedding learning problem with  $\text{sgn}(x)$  function as  $\delta \rightarrow \infty$ . In the optimization process, we can start setting a small  $\delta$  at first stage, such as  $\delta_0 = 1$ . Next, we increase the  $\delta$  to  $\delta_1$  ( $\delta_1 > \delta_0$ ), initialize all the parameters by the results of the last step and train the model until convergence (i.e. fine-tune). By repeating this process, we will obtain the exactly binary embeddings by increasing  $\delta \rightarrow \infty$ . **In real experiments, we generally achieve quick convergence by 10 rounds training.**

### C. Search Incomplete KGs in Hamming Space

Given an input query  $q$ , the embedding  $\mathbf{q}$ , i.e., the location of  $q$  in the Hamming space, can be predicted by the model. Then, we can use the **bitwise operations (XNOR and Bitcount) to perform a fast linear scan** by comparing the Hamming distance between query embedding  $\mathbf{q}$  and each KG entity embedding  $\mathbf{e}$  for the nearest neighbor search. Usually, the search result is a small set of candidate entities with the same and smallest distances. **We can directly return these entities to users as search results or perform a further ranking for more specific queries: comparing the retrieved nearest neighbor entities with the input query according to the Euclidean distance or dot-product in continuous and dense embedding space.** The experimental results show that we can already achieve fast KG searching and obtain good results by the computation of the Hamming distance.



TABLE I  
SUMMARY STATISTICS FOR BIO AND FB15K DATASETS.

|             | Bio     |          |          | FB15k     |          |           |
|-------------|---------|----------|----------|-----------|----------|-----------|
|             | Train   | Validate | Test     | Train     | Validate | Test      |
| #One-triple | 304,645 | 49,915   | 58,966   | 6,516,946 | 80,367   | 724,115   |
| #Entities   | 162,632 | -        | -        | 14,950    | -        | -         |
| #Relations  | 56      | -        | -        | 1,344     | -        | -         |
| #2-triple   | 100,017 | 1,000/QT | 3,000/QT | 1,000,000 | 1000/QT  | 1,0000/QT |
| #3-triple   | 99,273  | 1,000/QT | 3,000/QT | 1,000,000 | 1000/QT  | 1,0000/QT |

("x-triple" means the query-answer pairs whose queries have  $x$  triples.  
"NUM/QT" indicates the number of query-answer pairs per query type.)

#### IV. EXPERIMENTS

In this section, we attempt to understand the relative performance of our proposed framework. In particular, we wish to answer i)"**is the proposed model efficient? does it scale to real-world incomplete KGs?**" Given this, the next question naturally raised would be ii)"**how well the searching results generated by our method?**". The experimental details are described as follows.

**Datasets:** Experiments in this paper were performed on two real-world KGs, i.e., biological interaction (Bio) [4] and Freebase 15k (FB15k) [17].

**Query answer pairs:** We follow the strategy in [4] to collect 7 different types query-answer pairs from the given KGs. The number for sampled queries for training/validation/testing are shown in Table I.

**Baselines:** Two KG embedding based searching approaches with TransE [17] were employed as our baselines:

- GQE [4] is an end-to-end model to answer graph queries in continuous embedding space with two geometric operations. It is the state-of-the-art method comparing the pipeline model proposed in [3] which separates the KG embedding learning and query answering.
- Simple-sign is a simple version with directly sign function on the learned embeddings of KG entities and queries by GQE [4]. It also ignores the hierarchical types information of entities.

##### A. Efficiency

Our learning-to-hash model can dramatically accelerate the similarity computation between embeddings of KG entities and queries. The average time costs of our method for answering a given query are  $3 \times 10^{-5}$  seconds on Bio and  $2 \times 10^{-5}$  seconds on FB15k respectively. We further performed the nearest neighbor search for 7 query types by both GQE and our method on two different KGs. Figure 4 illustrates clearly shows that our method's superior performance to the baseline:

- The average amount of time by our method is approximately 100 times faster than the dense embeddings based baseline GQE, which is very impressive. The key reason is that bitwise operations in the Hamming space provide significant speed-up compared to multiply-accumulate operations in the continuous vector space.

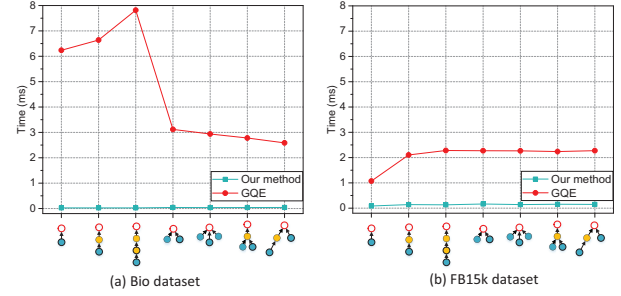


Fig. 4. Time costs of the GQE and our method on Bio and FB15k datasets.

- Our method achieves consistently improvements on different query types compared with the baseline, which demonstrates the scalability of our model.

For KGs with a larger size, we can gain more speedup by exploiting multi-index hashing [20] which can accelerate the entity searching by two orders of magnitude, since it has sub-linear run-time behavior for uniformly distributed embeddings.

##### B. Effectiveness

In this part, we evaluate the effectiveness of our method on different KGs. Given a query  $q$ , we aim to verify whether the discovered entity  $e$  satisfies query (i.e.,  $e \in \llbracket q \rrbracket$ ) compared to the negative example entities. Following the evaluation metrics in [4], we evaluate our model and baselines by the ROC AUC score and average percentile rank (APR). Figure 5 shows the detailed evaluation results of each query type.

- Comparing our method with GQE, results show that binary representations generated by learning-to-hash models can achieve competitive searching results like real-valued embeddings.
- Comparing our method with Simple-sign, we can see that our method obtain more lossless binary embeddings benefiting from the continuation method.
- Experimental results also indicate that incorporating the KG types ontology information is helpful to improve the model performance.

##### C. Parameter Sensitivity

We analyze the parameter sensitivity of binary embedding length  $d$  and tanh function coefficient  $\delta$ . Due to the limited space, we mainly demonstrate the affected results of AUC scores and hash rate on the Bio dataset. For the ACU scores, we can observe an interesting phenomenon that the averaged ACU scores only increased slightly when increasing the dimension of binary embeddings. More importantly, it indicates that the performance of binary embeddings learning is stable when the dimension is in the range from 64 to 512. For comparison purposes, we finally set the dimension 128 as same as the best configurations in GQE [4]. To verify that each bit of our embeddings is generally getting closer to +1 or -1 while increasing the value of  $\delta$ , we count the hash rate of embedding bits whose absolute values are larger than 0.9, as shown in the histogram Figure 6(b). We can observe that, when increasing

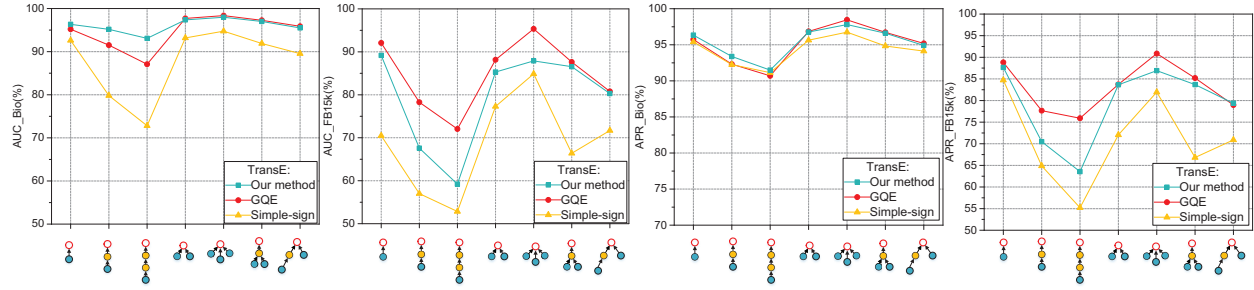


Fig. 5. AUC and APR scores of the TransE based querying models per query type on Bio dataset and FB15k dataset.

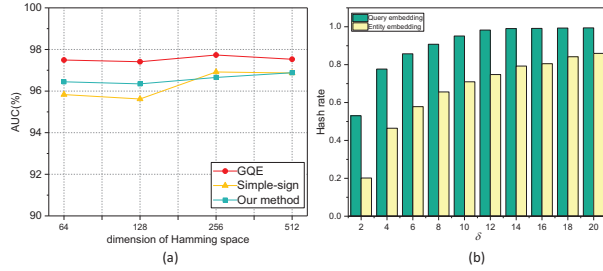


Fig. 6. Parameter Effects of the binary embedding length  $d$  and the tanh function coefficient  $\delta$  on Bio dataset.

the value of parameter  $\delta$  in training process, each bit of query embeddings or entity embeddings gradually gets close to +1 or -1. When  $\delta = 20$ , the hash rates are already 0.97 and 0.89 for query embeddings and entity embeddings respectively. Then, we can further directly apply the sign function on these embeddings to get the exactly hash codes. Considering the good performance on the quality of search results, this implies that our method address the ill-posed gradient problem for the binarization whilst embeddings keep nearly lossless.

## V. CONCLUSIONS AND FUTURE WORK

In this paper, we propose an end-to-end framework for searching incomplete KGs based on learning-to-hash. We design an end-to-end binary embedding model to preserve the triple structure information of graph queries and KG. Moreover, we utilize a continuation method with convergence guarantees to address the ill-posed gradient issue in the optimization. Experimental results show that the proposed framework is effective and efficient for speeding up the searching process and providing good answers than conventional dense vector embedding based models for unanswerable queries. In the future, we intend to propose a unified framework to jointly search incomplete KGs in hybrid spaces, i.g., original KG, dense vector space, and Hamming space.

## ACKNOWLEDGMENT

This work was supported by National Science Foundation of China with Grant Nos. 61906037 and U1736204; National Key R&D Program of China with Grant Nos. 2018YFC0830200 and 2017YFB1002801; the Judicial Big Data Research Centre, School of Law at Southeast University.

## REFERENCES

- [1] D. Vrandečić and M. Krötzsch, "Wikidata: a free collaborative knowledgebase," *Communications of the ACM*, vol. 57, no. 10, pp. 78–85, 2014.
- [2] K. Bollacker, C. Evans, P. Paritosh, T. Sturge, and J. Taylor, "Freebase: A collaboratively created graph database for structuring human knowledge," in *SIGMOD*. ACM, 2008, pp. 1247–1250.
- [3] M. Wang, R. Wang, J. Liu, Y. Chen, L. Zhang, and G. Qi, "Towards empty answers in sparql: Approximating querying with rdf embedding," in *ISWC*. Springer, 2018, pp. 513–529.
- [4] W. Hamilton, P. Bajaj, M. Zitnik, D. Jurafsky, and J. Leskovec, "Embedding logical queries on knowledge graphs," in *NeurIPS*, 2018, pp. 2026–2037.
- [5] Q. Wang, Z. Mao, B. Wang, and L. Guo, "Knowledge graph embedding: A survey of approaches and applications," *TKDE*, vol. 29, no. 12, pp. 2724–2743, 2017.
- [6] W. Li, Y. Zhang, Y. Sun, W. Wang, M. Li, W. Zhang, and X. Lin, "Approximate nearest neighbor search on high dimensional data-experiments, analyses, and improvement," *TKDE*, 2019.
- [7] J. Wang, T. Zhang, N. Sebe, H. T. Shen *et al.*, "A survey on learning to hash," *TPAMI*, vol. 40, no. 4, pp. 769–790, 2017.
- [8] Z. Cao, M. Long, J. Wang, and P. S. Yu, "Hashnet: Deep learning to hash by continuation," in *CVPR*. IEEE, 2017, pp. 5608–5617.
- [9] A. Gionis, P. Indyk, R. Motwani *et al.*, "Similarity search in high dimensions via hashing," in *PVLDB*, vol. 99, no. 6. VLDB Endowment, 1999, pp. 518–529.
- [10] H. Jégou, M. Douze, C. Schmid, and P. Pérez, "Aggregating local descriptors into a compact image representation," in *CVPR*. IEEE, 2010, pp. 3304–3311.
- [11] J. Sánchez and F. Perronnin, "High-dimensional signature compression for large-scale image classification," in *CVPR*. IEEE, 2011, pp. 1665–1672.
- [12] D. Lian, K. Zheng, V. W. Zheng, Y. Ge, L. Cao, I. W. Tsang, and X. Xie, "High-order proximity preserving information network hashing," in *SIGKDD*. ACM, 2018, pp. 1744–1753.
- [13] H. Yang, S. Pan, P. Zhang, L. Chen, D. Lian, and C. Zhang, "Binarized attributed network embedding," in *ICDM*. IEEE, 2018, pp. 1476–1481.
- [14] K. Kishimoto, K. Hayashi, G. Akai, M. Shimbo, and K. Komatani, "Binarized knowledge graph embeddings," in *ECIR*. Springer, 2019, pp. 181–196.
- [15] P. Ristoski and H. Paulheim, "Rdf2vec: Rdf graph embeddings for data mining," in *ISWC*. Springer, 2016, pp. 498–514.
- [16] M. Nickel, V. Tresp, and H.-P. Kriegel, "A three-way model for collective learning on multi-relational data," in *ICML*. Omnipress, 2011, pp. 809–816.
- [17] A. Bordes, N. Usunier, A. Garcia-Duran, J. Weston, and O. Yakhnenko, "Translating embeddings for modeling multi-relational data," in *NeurIPS*, 2013, pp. 2787–2795.
- [18] C. A. Hurtado, A. Poulouvasilis, and P. T. Wood, "Ranking approximate answers to semantic web queries," in *ESWC*. Springer, 2009, pp. 263–277.
- [19] R. Xie, Z. Liu, and M. Sun, "Representation learning of knowledge graphs with hierarchical types," in *IJCAI*, 2016, pp. 2965–2971.
- [20] M. Norouzi, A. Punjani, and D. J. Fleet, "Fast search in hamming space with multi-index hashing," in *CVPR*. IEEE, 2012, pp. 3108–3115.